ReasonPath: SLM Fine-tuning with Reasoning Approach for Web Path Prediction*

Kinsey K.S. Ng, Farah Yan, and Kevin Hung

School of Science and Technology, Hong Kong Metropolitan University, Hong Kong
{s1350047, fyan, khung}@hkmu.edu.hk

Abstract

Web path discovery is a critical reconnaissance step in web security testing. Traditional methods rely heavily on exhaustive dictionary-based fuzzing, which is resource-intensive and generates large volumes of traffic that can trigger defensive alerts on target systems. This paper presents ReasonPath, a novel approach that leverages reasoning-enhanced fine-tuning of Small Language Models (SLM) to improve the accuracy and efficiency of web path prediction for security testing while maintaining reasonable resource usage. By fine-tuning SLMs with chain-of-thought reasoning traces, the proposed method internalizes path pattern knowledge and generates more targeted candidates. The experimental results show that small models have high potential to achieve comparable performance to large model. ReasonPath improves the accuracy and efficiency of base SLMs. F1 is improved from near-zero (0.0002) to 0.29 for the best-performing 3B model and to 0.26 for the 0.5B model. Among the selected model, the Qwen2.5-0.5B has outperformed all other models with low peak VRAM.

1 Introduction

Web applications form the backbone of modern e-business systems, providing interactive services, data processing, and content delivery across diverse domains. As modern enterprises increasingly rely on web-facing platforms for transactions, customer data management, and supply-chain integration, safeguarding these infrastructures against unauthorized access and path exposure has become a critical priority. However, the complexity and scale of web applications introduce numerous security vulnerabilities, particularly in access control. Web applications often contain sensitive business information, including customer data, financial records, and proprietary assets, making them prime targets for attackers. According to the Verizon 2024 Data Breach Investigations Report, web-based attacks account for the majority of security incidents, underscoring the critical need for effective vulnerability assessment [1]. The OWASP Top 10 identifies broken access control and injection vulnerabilities as the most critical web application security risks [2]. The identification of sensitive paths and endpoints is a key part to assess vulnerabilities. Currently, traditional discovery methods have significant limitations in terms of both efficiency and coverage.

1.1 Motivation

Traditional web path discovery relies primarily on dictionary-based brute-force methods using tools such as Gobuster [3], Wfuzz [4], and Feroxbuster [5]. While effective, these approaches require extensive wordlists and generate excessive false positives through brute-force probing.

*Proceedings of The 2026 IFIP WG 8.4 International Symposium on E-Business Information Systems Evolution (EBISION 2026), Article No. 3, July 8-10, 2026, Seoul, South Korea.
© The copyright of this paper remains with the author(s).

Recent advances in reasoning-capable large language models give new insights in web path prediction. Models such as DeepSeek-R1 [6] and OpenAI o1 [7] demonstrate that reasoning-enhanced training can significantly improve performance on complex tasks requiring multi-step inference. So far, these capabilities have not been systematically applied to web path prediction for security testing.

However, LLM is too heavy for web path prediction task. While large LLMs demonstrate strong reasoning capabilities, the computational requirements and latency of such models make them impractical for high-volume security testing tasks. In particular, VRAM (Video Random Access Memory) requirements scale approximately linearly with parameter count: a 70B-parameter model such as Llama-2-70B requires approximately 140 GiB for inference in half-precision format [8], whereas a 3B-parameter model such as Qwen2.5-3B demands only 6 to 8 GiB [9], and a 0.5B-parameter model requires less than 2 GiB. This hardware constraint limits deployment on commodity GPUs, making Small Language Models (SLM) with 1-7B parameters a promising alternative [10], provided they undergo careful fine-tuning to develop reasoning capabilities comparable to LLMs.

1.2 Contributions

The present work addresses the research questions of whether reasoning-enhanced fine-tuning improves the web path prediction accuracy of SLMs compared with their non-fine-tuned counterparts, and whether a 0.5B-parameter SLM can achieve URL-sensitive path prediction with relevant performance. It makes four main contributions. Firstly, it proposes ReasonPath, a reasoning-enhanced fine-tuning approach for web path prediction using SLMs. Secondly, it designs a tailored fine-tuning pipeline. Thirdly, it constructs a dataset of 216,821 reasoning-annotated paths. Fourthly, it provides empirical evidence that fine-tuned SLMs lift F1 from near-zero to 0.29, with the 0.5B model reaching 0.26 F1 at only 1.4 GiB peak VRAM.

2 Related Work

2.1 Web Path Discovery

Web path discovery, also known as directory enumeration or endpoint discovery, is a fundamental reconnaissance technique in web security testing. Traditional approaches employ dictionary-based brute-force methods using tools such as Gobuster [3], Wfuzz [4], DirBuster [11], and Feroxbuster [5]. These tools systematically probe target servers with predefined wordlists, analyzing HTTP response codes to identify valid paths. Recent preliminary studies further demonstrate that traditional dictionary-based fuzzing achieves an accuracy of 0.030 and only 0.5 success hits per 100 requests on large-scale web corpora, with the overwhelming majority of probes returning non-existent paths [10].

The effectiveness of dictionary-based approaches depends heavily on wordlist quality and coverage. Common wordlists include SecLists [12] and custom curated lists. However, these static dictionaries suffer from limited coverage of application-specific paths and generate excessive false positives, requiring substantial probing budgets.

Recent work has explored clustering-based approaches to organize and prioritize path candidates. However, these methods still rely on static dictionaries and lack adaptive generation capabilities [13].

2.2 LLMs for Security Applications

Large language models have been increasingly applied to security tasks, including vulnerability detection, penetration testing, and security code review. LLM4Vuln [14] provided a unified evaluation framework for assessing LLM capabilities in vulnerability reasoning, identifying both strengths and limitations in security-specific tasks.

PentestAgent [15] introduced an LLM agent-based framework for automated penetration testing, incorporating retrieval-augmented generation to enhance the model's security knowledge. The system demonstrated that LLMs can effectively coordinate penetration testing workflows when provided with appropriate context and tools.

Fuzz4All [16] explored universal fuzzing with LLMs, showing that domain-specific fine-tuning can improve input generation for various fuzzing targets. However, the work focused on input generation rather than path discovery.

Despite these advances, none of the existing works systematically apply fine-tuning language models to web path prediction for security testing. This gap motivates the present work on ReasonPath, which bridges reasoning-capable LLMs with security-focused path discovery through targeted fine-tuning of Small Language Models.

2.3 Reasoning LLMs and Fine-tuning

The emergence of reasoning-capable LLMs has opened new possibilities for complex task solving. DeepSeek-R1 [6] demonstrated that reinforcement learning can incentivize reasoning capabilities, achieving state-of-the-art performance on mathematical and logical reasoning benchmarks. OpenAI's o1 series further advanced reasoning through test-time computation scaling.

Chain-of-thought fine-tuning has emerged as an effective technique for instilling reasoning capabilities. Kim et al. [17] showed that CoT fine-tuning improves performance on Big-Bench Hard by 2-4%. Recent work on diverse chain-of-thought fine-tuning [18] demonstrated that generating multiple reasoning chains within a single inference step enables within-inference refinement.

Process supervision, which provides feedback on intermediate reasoning steps rather than only final outcomes, has shown particular promise. Lightman et al. [19] demonstrated that process supervision significantly improves mathematical reasoning compared to outcome supervision alone.

Recent surveys [20,21] have comprehensively reviewed advances in reasoning models, categorizing techniques such as chain-of-thought prompting, process supervision, and diverse reasoning-chain generation. These surveys emphasize that domain-specific fine-tuning can transfer general reasoning capabilities to specialized tasks by aligning intermediate reasoning steps with task-specific structure, thereby bridging the gap between broad linguistic competence and targeted application performance.

3 Methodologies

3.1 Overview

ReasonPath employs a reasoning-enhanced fine-tuning approach to improve web path prediction. Figure 1 showing the overview of the ReasonPath methodology. The methodology consists of three stages: dataset construction with reasoning traces, chain-of-thought fine-tuning with process supervision, and directed path generation.

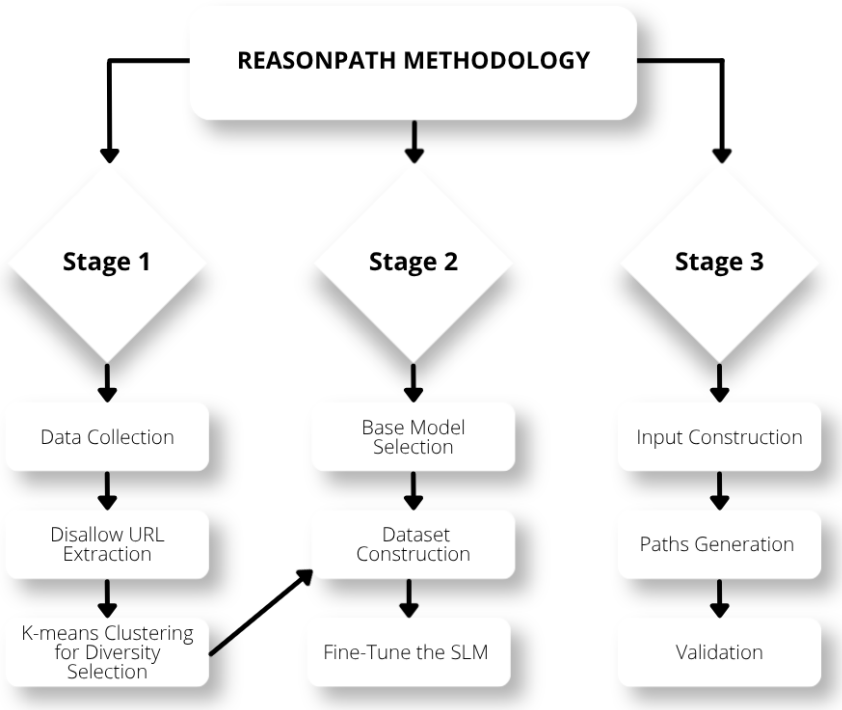


Figure 1: The ReasonPath framework overview showing the three-stage pipeline: dataset construction with reasoning traces, chain-of-thought fine-tuning, and directed path generation.

3.2 Stage 1: Dataset Construction with Reasoning Traces

The foundation of reasoning-enhanced fine-tuning is a high-quality dataset containing not only input-output pairs but also explicit reasoning traces that explain how outputs are derived from inputs.

3.2.1 Data Collection and robots.txt Crawling

Web path data are collected from multiple sources following the methodology established in a preliminary study [10]. A large-scale corpus of robots.txt files is gathered from Common Crawl [22] web crawl data. The Common Crawl Foundation provides open access to petabytes of web crawl data, including robots.txt files from millions of domains. The collection process targets all available robots.txt files from the Common Crawl archives spanning 2020 to 2024, processing 2,902,837 robots.txt files.

Dataset Statistics Table 1 summarizes robots.txt processing statistics from Common Crawl data.

Table 1: Robots.txt Dataset Statistics

Item	Value
Files processed	2,902,837
Total disallow URLs	52,811,644
Processing success rate	77.1%

3.2.2 Disallowed URL Extraction

Each robots.txt file is parsed to extract `Disallow` directives, which represent paths that website administrators explicitly mark as restricted from search engine indexing. These disallowed paths serve as high-quality ground-truth signals for sensitive or administrative paths. The extraction process normalizes paths by removing leading slashes and filtering empty or root-only entries (`/`). Each robots.txt record yields an average of 21.08 disallow URLs.

3.2.3 K-means Clustering for Diversity Selection

To ensure diversity in the selected robots.txt variants and avoid over-sampling highly repeated templates, K-means clustering is applied to the extracted disallow URL patterns. Each robots.txt file is represented as a TF-IDF vectorized document based on its disallow URL tokens. The clustering process groups robots.txt files with similar access control patterns, enabling selection of representative variants from each cluster rather than random sampling.

3.2.4 Representative Variant Selection

From each cluster, representative robots.txt variants are selected based on cluster centroid proximity. The top 10,000 distinct variants are chosen to maximize cluster coverage while minimizing redundancy. This approach ensures that the selected dataset captures diverse website structures and access control patterns across different domains, industries, and technology stacks.

3.2.5 LLM-Powered CoT Dataset Construction

To generate high-quality Chain-of-Thought (CoT) training examples, GPT-4o-mini is prompted to analyze actual website homepage structures and predict hidden robots.txt paths via a four-step reasoning chain (signature extraction, security-gap analysis, path prediction, and synthesis). For each domain, observed paths are first extracted from the homepage HTML to provide signals about content organization, technology stack, and purpose.

The LLM then analyzes these URL patterns to infer a website signature covering type, technology stack, content types, and primary function. Based on this signature, the model identifies typical sensitive functionality categories such as administrative interfaces, configuration files, and backup archives.

For each predicted hidden path, GPT-4o-mini provides explicit reasoning (for example, predicting `/admin/` as an administrative interface or `/wp-config.php` as a configuration file containing credentials). Each analysis generates 500 to 800 tokens of reasoning text, producing an input-output format that maps from website analysis and signature extraction to ranked sensitive-path predictions.

After homepage crawling, chain-of-thought generation, and quality filtering, the final fine-tuning dataset included 6,366 valid domains with 216,821 associated reasoning traces. This dataset forms the foundation for reasoning-enhanced fine-tuning described in Section 3.3. Table 2 shows reasoning-annotated path statistics across selected web domains.

Table 2: Multi-Path CoT Dataset Statistics

Item	Value
Total domains	6,366
Total paths	216,821
Avg paths per domain	34.1
Median paths per domain	22
Min paths per domain	2
Max paths per domain	1,173
Quality filters	Length $\geq$3, no wildcards, no query params

Figure 2 shows two representative examples across different website types: WordPress CMS and E-Business Website. Each example demonstrates how reasoning traces guide multi-path prediction based on detected technology patterns.

Figure 2: Reasoning Trace Examples

WordPress (PHP)	**E-Business Website**
Input: example.com Tech: WordPress, Apache, PHP	Input: shop.example.com Tech: Node.js, React, PostgreSQL
Reasoning: 1. WordPress uses /wp-admin/ 2. Uploads in /wp-content/ 3. Backups: /backup.sql 4. Config: /wp-config.php.bak	Reasoning: 1. E-business uses /admin/ 2. API at /api/v1/ 3. Orders: /orders/export 4. Reports: /reports/sales
Output: /wp-admin/ /wp-content/uploads/ /backup.sql /wp-config.php.bak	Output: /admin/login /api/v1/orders /orders/export.csv /reports/sales.pdf

3.3 Stage 2: Chain-of-Thought Fine-tuning

Base language models are fine-tuned on the constructed dataset using supervised fine-tuning on chain-of-thought reasoning traces. The training pipeline is implemented in Python using the HuggingFace Transformers ecosystem, with model configuration, dataset formatting, and training orchestration modularized across dedicated components.

3.3.1 Base Model Selection

Table 3 lists selected small language models and the selection reasons.

Table 3: Base Model Selection

Model	Params	Selection Rationale
Qwen2.5-3B-Instruct	∼3B	Strong instruction-following and multilingual open-source SLM
DeepSeek-R1-Distill-Qwen-1.5B	∼1.5B	Distilled from 671B teacher on 800K reasoning samples; strong logic/math per parameter
Phi-3-mini-4k-instruct	∼3.8B	High-quality training; outperforms larger baselines on reasoning and coding benchmarks
Qwen2.5-0.5B-Instruct	∼0.5B	Grouped-query attention edge SLM for sub-2 GiB mobile and IoT deployment

These four models span the 0.5B to 3.8B parameter range representative of contemporary Small Language Models. They were chosen to isolate how model size and VRAM constraints interact with reasoning-enhanced fine-tuning, and to test whether sub-1B models can compensate for smaller capacity via structured chain-of-thought supervision.

3.3.2 Prompt Engineering and Dataset Formatting

Each raw JSON sample is transformed into a chat message triplet (system, user, assistant) via a dedicated dataset builder module. The system message establishes the cybersecurity assistant role, while the user message concatenates the detected technology stack, observed homepage paths, and sitemap contents. The assistant message interleaves the reasoning trace with the structured list of predicted sensitive paths.

Model-specific formatting adaptations are applied at this stage. For the DeepSeek-R1 distill model, reasoning traces are wrapped in `<think>...</think>` tags to align with its trained reasoning format; for all other models, reasoning is emitted as free-form text preceding the path list. This conditional formatting ensures that the supervision signal matches each model's native inference distribution. The formatted records are then truncated to a maximum sequence length of 2048 tokens, and split into 95% training and 5% validation sets.

3.4 Stage 3: Directed Path Generation

The fine-tuned model then generates paths for validating the performance. For each target domain, an input prompt is constructed containing the target domain name, detected technologies, and observed paths from initial crawling, formatted with the model's chat template which establishes the cybersecurity assistant role. During validation, because each domain contains a variable amount of observed information and the model autoregressively determines when to stop generating, the number of predicted paths varies per domain, capped at a maximum of 20 paths. Paths are extracted from the output, normalized (lowercased, leading slash ensured, trailing slashes stripped), and deduplicated.

4 Experimental Settings

Evaluation is performed on a held-out validation split comprising 5% of the full dataset, with the same evaluation sets used across all models. Domain validation begins by constructing prompts with URLs from homepage discovery, inputting them into both the fine-tuned and non-fine-tuned models, and then comparing the generated URLs against the existing dataset's known sensitive URL sets. Path-level string matching is employed, and the comparison is domain-based because each domain, due to the theme of the website, may contain specific terms that vary from others. Predicted paths are normalized (lowercased, leading slash ensured, trailing slashes stripped).

Metrics are macro-averaged across each domain. Let D denote the set of validation domains. For each domain $d \in D$, precision P_d and recall R_d are computed from the domain's true positives (TP_d), false positives (FP_d), and false negatives (FN_d):

$$P_d = \frac{\mathrm{TP}_d}{\mathrm{TP}_d + \mathrm{FP}_d}, \qquad R_d = \frac{\mathrm{TP}_d}{\mathrm{TP}_d + \mathrm{FN}_d}.$$

Domain-level F1 is the harmonic mean of P_d and R_d:

$$\mathrm{F1}_d = 2 \cdot \frac{P_d \cdot R_d}{P_d + R_d}.$$

Precision, recall, and F1 are then obtained by averaging the per-domain values:

$$\mathrm{Precision}_M = \frac{1}{|D|} \sum_{d \in D} P_d, \qquad \mathrm{Recall}_M = \frac{1}{|D|} \sum_{d \in D} R_d, \qquad \mathrm{F1}_M = \frac{1}{|D|} \sum_{d \in D} \mathrm{F1}_d.$$

Also, peak VRAM is measured during inference by loading the model, and recording the maximum memory allocated.

5 Results

5.1 Comparison with Base Models

ReasonPath achieves improved performance across all fine-tuned models. Qwen2.5-3B attains the highest F1 (0.2935). Notably, Qwen2.5-0.5B achieves an F1 of 0.2612, within 11% of the 3B model, while requiring only 1,456 MiB of peak VRAM, a 5.4× reduction compared to the 3B model and a 6.5× reduction compared to Phi-3-mini. This makes the 0.5B parameter model highly attractive for edge and low-resource deployment scenarios.

All four base models achieve near-zero F1 (<0.0011) because they fail to produce outputs in the required output format. After fine-tuning, Qwen2.5-3B improves from 0.0002 to 0.2935 and Qwen2.5-0.5B improves from 0.0011 to 0.2612, demonstrating that structured reasoning supervision dramatically enhances task performance across model architectures of different parameter sizes.

Table 4 compares fine-tuned and base model performance metrics.

Table 4: Fine-Tuned versus Base Model Results ($n = 319$ validation domains)

Method	Peak VRAM	F1	TP	FP	FN
ReasonPath (Qwen2.5-3B FT)	7,812 MiB	**0.2935**	1,582	1,265	8,904
ReasonPath (DeepSeek-R1-Qwen-1.5B FT)	5,128 MiB	0.1401	1,032	1,385	9,454
ReasonPath (Phi-3-mini FT)	9,462 MiB	0.1196	578	727	9,908
ReasonPath (Qwen2.5-0.5B FT)	**1,456 MiB**	0.2612	1,418	1,553	8,877
Base SLM (Qwen2.5-3B)	7,590 MiB	0.0002	1	1,899	10,348
Base SLM (DeepSeek-R1-Qwen-1.5B)	5,018 MiB	0.0000	0	2	10,349
Base SLM (Phi-3-mini)	9,126 MiB	0.0005	3	25	10,346
Base SLM (Qwen2.5-0.5B)	1,423 MiB	0.0011	6	1,170	10,303
Traditional dict. fuzzing (EBISION)	—	0.027	—	—	—

6 Discussion

6.1 Performance of Reasoning Fine-tuning

Reasoning-enhanced fine-tuning outperforms base-model baselines. Internalized pattern knowledge enables faster inference. The chain-of-thought enhanced by fine-tuning on the given dataset ensures that reasoning steps are logically coherent, and improves accuracy over both non-fine-tuned models and traditional brute-force methods.

The train/validation/test split in the current pipeline is performed at the path level rather than at the domain level, meaning that all types of paths from different sources, such as `/wp-admin/`, `/administrator/`, or `/api/`, may appear across all partitions. The research consider unseen-domain discovery to be future work and recommend adopting domain-disjoint methods.

The current evaluation compares fine-tuned models against their zero-shot baselines. Further comparison with a direct-supervised baseline trained to predict paths without intermediate chain-of-thought traces could also be considered in the future. Such a baseline would help clarify whether the F1 improvement is attributable to the reasoning structure itself or simply to additional supervised exposure to path-label pairs.

6.2 VRAM and Model-Size Trade-offs

Peak VRAM consumption varies substantially across model architectures and parameter scales. During fine-tuned inference, Qwen2.5-3B demands approximately 7.8 GiB, Phi-3-mini requires 9.5 GiB, and DeepSeek-R1-Qwen-1.5B consumes 5.1 GiB. In contrast, the Qwen2.5-0.5B model operates with only 1.4 GiB of peak VRAM, a 5.4× reduction relative to the 3B model and a 6.5× reduction relative to Phi-3-mini, while attaining an F1 score of 0.26, within 11% of the best-performing 3B checkpoint.

These results indicate that reasoning-enhanced fine-tuning can unlock practical accuracy without proportional increases in memory footprint. Although larger models may still yield marginal improvements, such gains are increasingly constrained by hardware availability and deployment environment limitations. Ultra-compact checkpoints (sub-1B parameters) are consequently easier to adopt on commodity hardware, edge devices, and shared computing environments where high-end GPUs are unavailable.

6.3 Dataset Quality

The dataset uses robots.txt `Disallow` directives as ground-truth signals for sensitive paths. Although this source is large-scale, not every disallowed path corresponds to a security-relevant endpoint; introducing label noise that should be acknowledged when interpreting metrics. Furthermore, the prevalence of common administrative templates (e.g., `/admin`, `/administrator`) may limit generalization to modern dynamic web applications and API-driven systems.

6.4 Security Implications

The improved accuracy of ReasonPath has dual-use implications. Security teams can use the tool for more efficient penetration testing and vulnerability assessment. However, adversaries could also leverage similar techniques for malicious reconnaissance. Responsible disclosure and access controls are recommended for deployed systems.

6.5 Ethical Considerations

Testing system vulnerabilities raises ethical concerns. While it advances authorized security assessments, misuse of this research risks enabling unauthorized reconnaissance. Testing on real production environments should require explicit authorization and rate-limiting to avoid disrupting live systems. Laboratory evaluations should follow the principle of minimizing system impact.

6.6 Limitations

Several limitations needs to be mentioned. The evaluation relies on a status-code mapping that treats 301/302 redirects as false positives, reflecting that redirections do not, by themselves, confirm the presence of the intended sensitive resource. In practice, however, many production stacks implement redirection logic at the application layer while returning HTTP 200, for example, via HTML meta-refresh tags or JavaScript-based routing. Likewise, "soft-404" pages may return HTTP 200 with templated "not found" content, so a purely status-based evaluation can misclassify client-side redirects and soft-404s as true positives.

Another limitation concerns data cost and model freshness: producing annotated chain-of-thought reasoning traces at scale is expensive, and the model's knowledge becomes static after fine-tuning. Unlike retrieval-augmented approaches that can incorporate new data dynamically without retraining, this method requires periodic regeneration of reasoning datasets, and long-context inference with larger teacher models elevates latency and cost during that generation.

6.7 Future Work

Scaling and Continual Learning Scaling the dataset generation pipeline to full RAG-based retrieval on GPU clusters would enable semantic similarity scoring across the entire 3.7M-file corpus rather than relying on pre-clustered variants, potentially improving path diversity and coverage. Online or continual fine-tuning mechanisms could also be explored so that newly discovered paths are incorporated dynamically without retraining from scratch.

Reducing False Negatives To reduce the high false-negative rate, future work could investigate top-k [23] expansion with self-consistency voting, iterative refinement loops that re-prompt

the model with missed paths, and retrieval-augmented generation to inject analogous domain signatures at inference time.

Beyond Traditional Web Paths The methodology can be extended beyond traditional web paths to API endpoint discovery, where RESTful and GraphQL endpoint patterns present a richer but structurally different prediction space. Integrating more sources from real-world URL patterns is also required to provide a more diverse dataset.

7 Conclusion

This study examined whether reasoning-enhanced fine-tuning improves URL path prediction in Small Language Models, and whether an extremely compact SLM can achieve relevant accuracy for this task. Experiments show that reasoning-aware adaptation yields dramatic improvements over vanilla checkpoints. Furthermore, accuracy gains scale sub-linearly with parameter count; the 0.5B checkpoint attains F1 within 11% of its 3B counterpart, indicating that larger architectures confer only marginal gains once structured reasoning training is provided. VRAM consumption, however, scales almost linearly with model size: the Qwen2.5-0.5B model requires only 1.4 GiB peak allocation, whereas the 3B model demands 5.4 times more VRAM for merely 3.2 percentage points of additional F1. These results confirm that sub-1B models can internalize meaningful path-pattern knowledge through adaptive reasoning, offering a practical balance of accuracy and efficiency for security assessment.

References

[1] Verizon Enterprise Solutions, "2024 data breach investigations report," Verizon Business, Tech. Rep., 2024, analysis of 30,458 security incidents and 10,626 confirmed data breaches across 94 countries. [Online]. Available: https://www.verizon.com/business/resources/reports/2024-dbir-data-breach-investigations-report.pdf

[2] OWASP Foundation, "Owasp top 10:2021," 2021, standard awareness document for web application security risks. [Online]. Available: https://owasp.org/Top10/2021/

[3] O. Reeves, "Gobuster: Directory/file, dns and vhost busting tool," 2015, directory/file, DNS and virtual host brute-forcing tool written in Go. [Online]. Available: https://github.com/OJ/gobuster

[4] X. Méndez, "Wfuzz: Web application fuzzer," 2010, web application brute-forcing tool for finding resources, injection testing, and parameter fuzzing. [Online]. Available: https://github.com/xmendez/wfuzz

[5] B. Davidson, "Feroxbuster: Fast recursive content discovery tool," 2020, fast, simple, recursive content discovery tool written in Rust. [Online]. Available: https://github.com/epi052/feroxbuster

[6] DeepSeek-AI, D. Guo, D. Yang, H. Zhang, J. Song, P. Wang, Q. Zhu, R. Xu, R. Zhang, S. Ma, X. Bi, X. Zhang, X. Yu, Y. Wu, Z. F. Wu, Z. Gou, Z. Shao, Z. Li, Z. Gao, A. Liu, B. Xue, B. Wang, B. Wu, B. Feng *et al.*, "Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning," *Nature*, vol. 645, no. 8081, pp. 633–638, 2025, demonstrates remarkable reasoning capabilities through large-scale reinforcement learning without supervised fine-tuning. [Online]. Available: https://www.nature.com/articles/s41586-025-09422-z

[7] OpenAI, A. Jaech, A. Kalai, A. Lerer, A. Richardson *et al.*, "Openai o1 system card," OpenAI, Tech. Rep., 2024, the o1 model series is trained with large-scale reinforcement learning to reason using chain of thought. [Online]. Available: https://cdn.openai.com/o1-system-card.pdf

[8] Micron Technology, "Accelerating large language model inference with hbm3e," Micron Technology, Tech. Rep., 2024, technical brief documenting approximate memory requirements for LLM inference. [Online]. Available: https://www.micron.com/content/dam/micron/global/public/documents/products/technical-marketing-brief/hbm3e-h200-technical-brief.pdf

[9] Qwen Team, "Qwen2.5 technical report," 2025, open-source language models with strong reasoning and instruction-following capabilities. [Online]. Available: https://huggingface.co/Qwen/Qwen2.5-7B-Instruct

[10] K. K. Ng, F. Yan, and K. Hung, "A preliminary study of retrieval-augmented generation for sensitive web path discovery in web-based information systems," in *EBISION 2025*. The 2025 IFIP WG 8.4 International Symposium on E-Business Information Systems Evolution, 2025, preliminary work on RAG-based approach for web path discovery using robots.txt databases and LangChain agents.

[11] OWASP Foundation, "Dirbuster: Directory and file enumeration tool," 2007, oWASP project for directory brute-forcing. [Online]. Available: https://www.kali.org/tools/dirbuster/

[12] Full Disclosure, "Seclists: Security mailing list archives and fuzzing wordlists," 2024, security mailing list archive and wordlist repository. [Online]. Available: https://seclists.org/

[13] M. Al-Hawawreh and E. Sitnikova, "Fuzzing frameworks for server-side web applications: a survey," *International Journal of Information Security*, vol. 23, no. 4, pp. 2897–2924, 2024. [Online]. Available: https://link.springer.com/article/10.1007/s10207-024-00979-w

[14] Y. Sun, D. Wu, Y. Xue, H. Liu, W. Ma, L. Zhang, Y. Liu, and Y. Li, "Llm4vuln: A unified evaluation framework for decoupling and enhancing llms' vulnerability reasoning," *arXiv preprint arXiv:2401.16185*, 2024. [Online]. Available: https://arxiv.org/abs/2401.16185

[15] X. Shen, L. Wang, Z. Li, Y. Chen, W. Zhao, D. Sun, J. Wang, and W. Ruan, "Pentestagent: Incorporating llm agents to automated penetration testing," in *Proceedings of the 20th ACM Asia Conference on Computer and Communications Security (ASIA CCS '25)*. ACM, 2025. [Online]. Available: https://dl.acm.org/doi/10.1145/3708821.3733882

[16] C. S. Xia, M. Paltenghi, J. L. Tian, M. Pradel, and L. Zhang, "Fuzz4all: Universal fuzzing with large language models," in *Proceedings of the 46th International Conference on Software Engineering (ICSE '24)*. ACM, 2024. [Online]. Available: https://dl.acm.org/doi/10.1145/3597503.3639121

[17] S. Kim, J. Shin, Y. Cho, M. Jang, S. Longpre, H. Lee, S. Yun, S. Park, and M. Kim, "The cot collection: Improving zero-shot and few-shot learning of language models via chain-of-thought fine-tuning," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023, pp. 1738–1750. [Online]. Available: https://aclanthology.org/2023.emnlp-main.782.pdf

[18] H. Puerto, T. Chubakov, X. Zhu, H. T. Madabushi, and I. Gurevych, "Fine-tuning on diverse reasoning chains drives within-inference cot refinement in llms," in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Vienna, Austria: Association for Computational Linguistics, July 2025, pp. 3789–3808. [Online]. Available: https://aclanthology.org/2025.acl-long.191/

[19] H. Lightman, V. Kosaraju, Y. Burda, H. Edwards, B. Baker, T. Lee, J. Leike, J. Schulman, I. Sutskever, and K. Cobbe, "Let's verify step by step," in *Proceedings of the Twelfth International Conference on Learning Representations (ICLR)*, 2024. [Online]. Available: https://openreview.net/forum?id=v8L0pN6EOi

[20] M. A. Ferrag, N. Tihanyi, and M. Debbah, "Reasoning beyond limits: Advances and open problems for llms," *ICT Express*, vol. 11, no. 6, pp. 1054–1096, 2025. [Online]. Available: https://doi.org/10.1016/j.icte.2025.09.003

[21] Y. Annepaka and P. Pakray, "Large language models: A survey of their development, capabilities, and applications," *Knowledge and Information Systems*, vol. 67, no. 3, pp. 2967–3022, 2025. [Online]. Available: https://doi.org/10.1007/s10115-024-02310-4

[22] Common Crawl Foundation, "Common crawl: Open repository of web crawl data," 2024, nonprofit 501(c)(3) organization providing open web crawl data. [Online]. Available: https://commoncrawl.org/

[23] A. Fan, M. Lewis, and Y. Dauphin, "Hierarchical neural story generation," in *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2018, introduced top-k sampling for neural text generation. [Online]. Available: https://aclanthology.org/P18-1082/